

AnomX — Isolation Forest Report

Seasons of Code 2026 — Suyash Jagtap (24b1251)

How Isolation Forest Works

Isolation Forest is basically trying to isolate each data point from the rest. The idea is that anomalous points are rare and different, so they are easy to isolate. Normal points are grouped together, so you need a lot more splits to separate them.

The algorithm builds a bunch of random decision trees. Each tree picks a random feature and a random split value, and keeps splitting until each point is alone. The number of splits it takes to isolate a point is called its path length.

If a point gets isolated in very few splits — it's probably an anomaly. If it takes a lot of splits — it's probably normal. This is averaged across all trees to get a final score.

ML Workflow Used

The steps I followed to run the model:

- Load `features.csv` — the dataset with 67 columns that we built in week 4
- Drop columns like `user_id`, `timestamp`, `anomaly_type` — these are not features, just labels or identifiers
- Keep only numeric columns — Isolation Forest needs numbers, not strings like country or device
- Train the model — fit it on the feature matrix with contamination set to 0.15 (since 15% of our users are anomalous)
- Get predictions — model returns +1 for normal and -1 for anomalous
- Evaluate — compare predictions with the actual `is_anomalous` column using precision, recall and F1 score

Why Feature Engineering Is Needed

Raw events don't give enough context. If I just feed `event_type` and `amount` into the model, it won't know if a \$15,000 trade is normal or suspicious — because it depends on who the user is and what they usually do.

Feature engineering creates columns that capture patterns over time. For example:

- `rolling_failed_attempts_5` — are failed logins building up? brute_forcer averages 4.04 here, normal users 0.03
- `roll_5_trade_vol_mean` — is trading volume suddenly much higher than usual? wash_trader averages 27,585 vs normal 3,524
- `unique_countries_last_10_logins` — is this user logging in from many different countries? ip_hopper hits 6–8, normal stays at 1–2

- `deposit_amount_rolling_std` — are all deposits suspiciously similar in size? `structurer` has very low std since all deposits are \$490–\$999

Without these, the model is basically blind to patterns. Features are what make the model actually useful.

How Anomaly Scores Are Generated

After training, the model scores every event based on its average path length across all trees. Short path = isolated quickly = more anomalous. The score is normalized between 0 and 1.

In scikit-learn, `predict()` gives a binary label (+1 or -1). But `decision_function()` gives the actual score, which is more useful because you can rank events from most suspicious to least suspicious instead of just getting a yes/no answer.

How Suspicious Events Are Identified

Events with predicted label -1 are flagged as suspicious. In our dataset the signals are pretty clearly separated by anomaly type:

- **brute_forcer** — `burst_count_5min` spikes and `rolling_failed_attempts_5` is high
- **ip_hopper** — `unique_countries_last_10_logins` reaches 6–8, `rolling_timezone_gap_5` is high
- **wash_trader** — `roll_5_trade_vol_mean` is 8x above normal, `pnl` always positive
- **structurer** — low `deposit_amount_rolling_std`, high `deposit_count_last_10`
- **bot_trader** — `roll_5_click_rate_mean` is 178/min vs 0.30 for normal users
- **dormant_withdrawer** — `is_large_withdrawal` flag triggered, high `withdrawal_amount_vs_account_balance`
- **deposit_withdrawal_cycler** — `rolling_immediate_withdrawal_5` > 0, `withdrawal_to_deposit_ratio` near 1

One thing I noticed is that just flagging events is not enough. You also need to know why it was flagged. In week 5–6 we will look at ways to explain model decisions so the output is actually useful and not just a list of flags.